

Lab 5 — Lexical Analysis

2023-CS-18 M Ahmed

March 16, 2026

1. Overview

In this lab we built a lexical analyzer for a subset of Pascal (from Dragon Book Appendix A). The job of a lexical analyzer is to read source code and break it into tokens — things like keywords, identifiers, numbers, and operators.

We implemented the same lexer three different ways so we could compare how each approach works and how fast each one runs.

2. Pascal Subset Tokens

The grammar gives us these token types:

Token	What it matches	Example
KEYWORD	reserved words	<code>begin, if, while</code>
ID	letter followed by letters/digits	<code>gcd, x, myVar</code>
NUM	integer or real number	<code>42, 3.14, 1.5E10</code>
ASSIGNOP	assignment	<code>:=</code>
RELOP	comparison	<code>=, <>, <, <=, >, >=</code>
ADDOP	addition level	<code>+, -, or</code>
MULOP	multiplication level	<code>*, /, div, mod, and</code>
DOTDOT	array range	<code>..</code>

Comments are written inside `{ }` and cannot be nested. Whitespace is ignored everywhere.

3. Approach 1 — State-Based DFA

This approach uses one loop and one `state` variable. Every character we read moves us to a new state. When the state tells us a token is complete, we return it.

For example: reading `:` moves us to state `COLON`. Then if the next character is `=` we return `ASSIGNOP :=`, otherwise we return `COLON :`.

The states we used are:

State	Meaning
START	waiting for first character
ID	reading an identifier
INT	reading integer digits
FRAC	reading decimal part
EXP	reading exponent
COLON	saw :, waiting to see if next is =
LT	saw <, waiting for = or >
GT	saw >, waiting for =
DOT	saw ., waiting for second .

4. Approach 2 — Stateless

This approach has no state variable at all. Instead, `next_token()` just looks at the first character and immediately calls the right helper function.

If it sees a letter it calls `scan_word()`. If it sees a digit it calls `scan_num()`. If it sees `:` it handles it inline. Each case is completely independent of the others.

This is the simplest approach to read and understand. Adding a new token type just means adding one new `if` line.

5. Approach 3 — Transition Table

This approach puts all the DFA transitions into a 2D table:

```
table[current_state][char_class] = next_state
```

Every character is first classified into a class (like `DIGIT`, `LETTER`, `EQ`, etc.). Then we just look up the table to find the next state. The loop itself has no `if/elif` logic for token types at all.

This is how real tools like `lex` and `flex` work. The advantage is that you can change the language just by changing the table — the driver loop stays the same.

Bonus — Compressed Table

The full table has $12 \times 22 = 264$ cells but only 58 of them are not error entries. So we also built a compressed version that stores only the 58 useful entries in a dictionary. This saves 78% of the memory with almost no speed difference.

6. Sample Output

We tested all approaches on the GCD program from Figure A.1 in the Dragon Book. All four produced exactly the same 68 tokens. Here are the first few lines:

Line	Type	Lexeme	Value
1	KEYWORD	program	-
1	ID	example	-
1	LPAREN	(	-
1	ID	input	-
1	COMMA	,	-
1	ID	output	-
1	RPAREN	)	-
1	SEMICOLON	;	-
2	KEYWORD	var	-
2	ID	x	-
...			
Total: 68 tokens			

7. Speed Comparison

We ran each lexer 1000 times on a 10,040 character source file (3,021 tokens) and measured the average time per run.

Rank	Approach	Time (ms)	vs Fastest
1	Approach 2 — Stateless	0.1455	1.00x
2	Approach 1 — State-Based DFA	0.1646	1.13x
3	Approach 3 — Transition Table	0.3033	2.08x
4	Bonus — Compressed Table	0.3241	2.23x

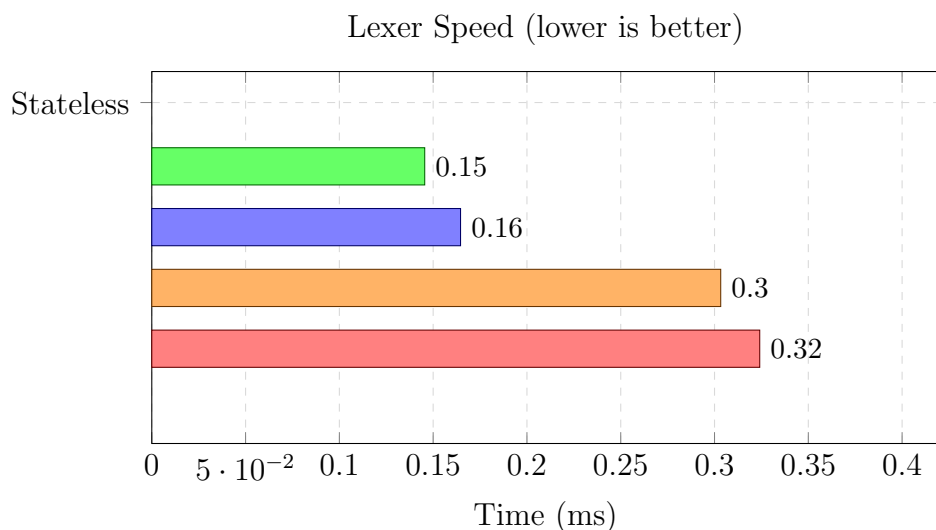


Figure 1: Average time per run for each approach (1000 runs)

Why Approach 2 is fastest: It uses plain if checks on one character with no extra overhead. Approach 3 is slower because every character needs two extra steps — classify it into a class, then do a table lookup. The compressed table is slightly slower than the full table because dictionary lookup is a bit slower than a direct list index.

8. Summary

	Approach 1	Approach 2	Approach 3
Speed	Fast	Fastest	Slow
Easy to read	Medium	Yes	No
Easy to change	Medium	Yes	Yes (edit table)
Auto-generate	No	No	Yes

Approach 2 is the best choice if you are writing a lexer by hand. Approach 3 is the right choice when using a tool that generates the lexer automatically from grammar rules.

0.1 Screenshot of Execution Approach 1

```
PASCAL SUBSET LEXER – ALL APPROACHES

Source:
program example(input, output);
var x, y: integer;
function gcd(a, b: integer): integer;
begin
    if b = 0 then gcd := a
    else gcd := gcd(b, a mod b)
end;
begin
    read(x, y);
    write(gcd(x, y))
end.
```

=====

Approach 1 – State-Based DFA

=====

Line	Type	Lexeme	Value
1	KEYWORD	program	-
1	ID	example	-
1	LPAREN	(	-
1	ID	input	-
1	COMMA	,	-
1	ID	output	-
1	RPAREN	)	-
1	SEMICOLON	;	-
2	KEYWORD	var	-
2	ID	x	-
2	COMMA	,	-
2	ID	y	-
2	COLON	:	-
2	KEYWORD	integer	-
2	SEMICOLON	;	-
3	KEYWORD	function	-
3	ID	gcd	-
3	LPAREN	(	-
3	ID	a	-
3	COMMA	,	-
3	ID	b	-
3	COLON	:	-
3	KEYWORD	integer	-
3	RPAREN	)	-
3	COLON	:	-
3	KEYWORD	integer	-
3	SEMICOLON	;	-
4	KEYWORD	begin	-
5	KEYWORD	if	-
5	ID	b	-

=====

```

9      COMMA      ,      -
9      ID         y      -
9      RPAREN    )      -
9      SEMICOLON ;      -
10     ID         write  -
10     LPAREN    (      -
10     ID         gcd    -
10     LPAREN    (      -
10     ID         x      -
10     COMMA      ,      -
10     ID         y      -
10     RPAREN    )      -
10     RPAREN    )      -
11     KEYWORD   end     -
11     DOT        .      -
12     EOF        EOF    -

```

```

-----
Total: 68 tokens
=====

```

0.2 Screenshot of Execution Approach 2

Approach 2 – Stateless			
Line	Type	Lexeme	Value
1	KEYWORD	program	-
1	ID	example	-
1	LPAREN	(	-
1	ID	input	-
1	COMMA	,	-
1	ID	output	-
1	RPAREN	)	-
1	SEMICOLON	;	-
2	KEYWORD	var	-
2	ID	x	-
2	COMMA	,	-
2	ID	y	-
2	COLON	:	-
2	KEYWORD	integer	-
2	SEMICOLON	;	-
3	KEYWORD	function	-
3	ID	gcd	-
3	LPAREN	(	-
3	ID	a	-
3	COMMA	,	-
3	ID	b	-
3	COLON	:	-
3	KEYWORD	integer	-
3	RPAREN	)	-
3	COLON	:	-
3	KEYWORD	integer	-
3	SEMICOLON	;	-
4	KEYWORD	begin	-
5	KEYWORD	if	-
5	ID	b	-
5	RELOP	=	-
5	NUM	0	0
5	KEYWORD	then	-
5	ID	gcd	-
5	ASSIGNOP	:=	-
5	ID	a	-
6	KEYWORD	else	-
6	ID	gcd	-
6	ASSIGNOP	:=	-
6	ID	gcd	-
6	LPAREN	(	-
6	ID	b	-
6	COMMA	,	-
6	ID	a	-
6	MULOP	mod	-
6	ID	b	-
6	RPAREN	)	-
7	KEYWORD	end	-
7	SEMICOLON	;	-
8	KEYWORD	begin	-
9	ID	read	-
9	LPAREN	(	-
9	ID	x	-
9	COMMA	,	-
9	ID	y	-
9	RPAREN	)	-
9	SEMICOLON	;	-

0.3 Screenshot of Execution Approach 3

Approach 3 – Transition Table			
Line	Type	Lexeme	Value
1	KEYWORD	program	-
1	ID	example	-
1	LPAREN	(	-
1	ID	input	-
1	COMMA	,	-
1	ID	output	-
1	RPAREN	)	-
1	SEMICOLON	;	-
2	KEYWORD	var	-
2	ID	x	-
2	COMMA	,	-
2	ID	y	-
2	COLON	:	-
2	KEYWORD	integer	-
2	SEMICOLON	;	-
3	KEYWORD	function	-
3	ID	gcd	-
3	LPAREN	(	-
3	ID	a	-
3	COMMA	,	-
3	ID	b	-
3	COLON	:	-
3	KEYWORD	integer	-
3	RPAREN	)	-
3	COLON	:	-
3	KEYWORD	integer	-
3	SEMICOLON	;	-
4	KEYWORD	begin	-
5	KEYWORD	if	-
5	ID	b	-
5	RELOP	=	-
5	NUM	0	0
5	KEYWORD	then	-
5	ID	gcd	-
5	ASSIGNOP	:=	-
5	ID	a	-
6	KEYWORD	else	-
6	ID	gcd	-
6	ASSIGNOP	:=	-
6	ID	gcd	-
6	LPAREN	(	-
6	ID	b	-
6	COMMA	,	-
6	ID	a	-
6	MULOP	mod	-
6	ID	b	-
6	RPAREN	)	-
7	KEYWORD	end	-
7	SEMICOLON	;	-
8	KEYWORD	begin	-
9	ID	read	-
9	LPAREN	(	-
9	ID	x	-
9	COMMA	,	-
9	ID	y	-
9	RPAREN	)	-
9	SEMICOLON	;	-
10	ID	write	-
10	LPAREN	(	-
10	ID	gcd	-
10	LPAREN	(	-
10	ID	x	-
10	COMMA	,	-
10	ID	y	-

0.4 Screenshot of Execution Approach 4

Approach 4 – Compressed Table			
Line	Type	Lexeme	Value
1	KEYWORD	program	-
1	ID	example	-
1	LPAREN	(	-
1	ID	input	-
1	COMMA	,	-
1	ID	output	-
1	RPAREN	)	-
1	SEMICOLON	;	-
2	KEYWORD	var	-
2	ID	x	-
2	COMMA	,	-
2	ID	y	-
2	COLON	:	-
2	KEYWORD	integer	-
2	SEMICOLON	;	-
3	KEYWORD	function	-
3	ID	gcd	-
3	LPAREN	(	-
3	ID	a	-
3	COMMA	,	-
3	ID	b	-
3	COLON	:	-
3	KEYWORD	integer	-
3	RPAREN	)	-
3	COLON	:	-
3	KEYWORD	integer	-
3	SEMICOLON	;	-
4	KEYWORD	begin	-
5	KEYWORD	if	-
5	ID	b	-
5	RELOP	=	-
5	NUM	0	0
5	KEYWORD	then	-
5	ID	gcd	-
5	ASSIGNOP	:=	-
5	ID	a	-
6	KEYWORD	else	-
6	ID	gcd	-
6	ASSIGNOP	:=	-
6	ID	gcd	-
6	LPAREN	(	-
6	ID	b	-
6	COMMA	,	-
6	ID	a	-
6	MULOP	mod	-
6	ID	b	-
6	RPAREN	)	-
7	KEYWORD	end	-
7	SEMICOLON	;	-
8	KEYWORD	begin	-
9	ID	read	-

0.5 Comparision

```
— Compressed Table Stats —————
Full table : 264 cells | Stored: 58 | Saved: 78%

— Consistency —————
OK Approach 1 – State-Based DFA
OK Approach 2 – Stateless
OK Approach 3 – Transition Table
OK Approach 4 – Compressed Table
All four produce identical token streams ✓

— Speed —————
0.1646 ms Approach 1 – State-Based DFA
0.1455 ms Approach 2 – Stateless
0.3033 ms Approach 3 – Transition Table
0.3241 ms Approach 4 – Compressed Table
```